

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Correlation and Pattern Analysis

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO4	Assessment:	Tool 3– Correlation and Pattern Analysis
Weightage:	25%	Total Marks:	
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.			
Student Name:	Sania Herbert	Reg. No.:	192324062
Section Batch: /	2023/AI&DS	Date of Submission:	17.8.26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

1. Time-Based Correlation Analysis

Given a dataset containing **monthly sales, advertising expenditure, customer visits, and product returns**, explore the data and identify correlations among the variables. Analyze time-related patterns, detect outliers, and present the findings using charts and an interactive visualization.

Python code:

```
# =====
# Q3: TIME-BASED CORRELATION ANALYSIS
# =====
# COMMAND 1: Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
print("Libraries imported successfully")
# =====
# COMMAND 2: Import Excel Dataset
# =====
df = pd.read_excel("Q3_Time_Based_Correlation_Analysis.xlsx")
print("Dataset imported successfully")
# =====
# COMMAND 3: Display First 5 Records
# =====
print("FIRST 5 RECORDS")
print(df.head())
# =====
# COMMAND 4: Display Last 5 Records
# =====
print("\nLAST 5 RECORDS")
print(df.tail())
# =====
# COMMAND 5: Display Dataset Shape
# =====
print("\nDATASET SHAPE")
print(df.shape)
# =====
# COMMAND 6: Display Column Names
# =====
print("\nCOLUMN NAMES")
print(df.columns.tolist())
# =====
# COMMAND 7: Display Dataset Information
# =====
print("\nDATASET INFORMATION")
df.info()
```

```

# =====
# COMMAND 8: Check Missing Values
# =====
print("\nMISSING VALUES")
print(df.isnull().sum())
# =====
# COMMAND 9: Check Duplicate Records
# =====
print("\nDUPLICATE RECORDS")
print("Number of duplicates:", df.duplicated().sum())
# =====
# COMMAND 10: Statistical Summary
# =====
print("\nSTATISTICAL SUMMARY")
print(df.describe())
# =====
# COMMAND 11: Convert Month to DateTime
# =====
df["Month"] = pd.to_datetime(df["Month"])
df = df.sort_values("Month")
print("\nMONTH COLUMN CONVERTED TO DATETIME")
print(df["Month"].head())
# =====
# COMMAND 12: Display Basic Business Statistics
# =====
print("\nBUSINESS STATISTICS")
print("Total Sales:",
      round(df["Sales"].sum(), 2))
print("Average Sales:",
      round(df["Sales"].mean(), 2))
print("Total Advertising Expenditure:",
      round(df["Advertising_Expenditure"].sum(), 2))
print("Average Customer Visits:",
      round(df["Customer_Visits"].mean(), 2))
print("Total Product Returns:",
      int(df["Product_Returns"].sum()))
# =====
# COMMAND 13: Monthly Sales Trend
# =====
plt.figure(figsize=(12, 5))
plt.plot(
    df["Month"],
    df["Sales"],
    marker="o"

plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()

```

```

plt.show(
# =====
# COMMAND 14: Advertising Expenditure Trend
# =====
plt.figure(figsize=(12, 5))
plt.plot(
    df["Month"],
    df["Advertising_Expenditure"],
    marker="o"
)
plt.title("Monthly Advertising Expenditure")
plt.xlabel("Month")
plt.ylabel("Advertising Expenditure")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# =====
# COMMAND 15: Customer Visits Trend
# =====
plt.figure(figsize=(12, 5))
plt.plot(
    df["Month"],
    df["Customer_Visits"],
    marker="o"
)
plt.title("Monthly Customer Visits")
plt.xlabel("Month")
plt.ylabel("Customer Visits")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# =====
# COMMAND 16: Product Returns Trend
# =====
plt.figure(figsize=(12, 5))
plt.plot(
    df["Month"],
    df["Product_Returns"],
    marker="o"
)
plt.title("Monthly Product Returns")
plt.xlabel("Month")
plt.ylabel("Product Returns")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# =====
# COMMAND 17: Combined Time-Based Visualization
# =====
fig, ax1 = plt.subplots(figsize=(12, 6))

```

```

ax1.plot(
    df["Month"],
    df["Sales"],
    marker="o",
    label="Sales"
)
ax1.set_xlabel("Month")
ax1.set_ylabel("Sales")
ax1.tick_params(axis="x", rotation=45)
plt.title("Monthly Business Performance")
plt.tight_layout()
plt.show()
# =====
# COMMAND 18: Calculate Correlation
# =====
correlation = df[
    "Sales",
    "Advertising_Expenditure",
    "Customer_Visits",
    "Product>Returns"
].corr()
print("\nCORRELATION MATRIX")
print(correlation)
# =====
# COMMAND 19: Correlation Heatmap
# =====
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
# =====
# COMMAND 20: Sales vs Advertising Expenditure
# =====
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="Advertising_Expenditure",
    y="Sales"
)
plt.title("Sales vs Advertising Expenditure")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")

```

```

plt.tight_layout()
plt.show()
# =====
# COMMAND 21: Sales vs Customer Visits
# =====
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="Customer_Visits",
    y="Sales"
)
plt.title("Sales vs Customer Visits")
plt.xlabel("Customer Visits")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
# =====
# COMMAND 22: Sales vs Product Returns
# =====
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="Product_Returns",
    y="Sales"
)
plt.title("Sales vs Product Returns")
plt.xlabel("Product Returns")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
# =====
# COMMAND 23: Outlier Detection Using Box Plot
# =====
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=df[
        [
            "Sales",
            "Advertising_Expenditure",
            "Customer_Visits",
            "Product_Returns"
        ]
    ]
)
plt.title("Outlier Detection")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

# =====

```

```
# COMMAND 24: Sales Outlier Detection Using IQR
```

```
# =====  
Q1 = df["Sales"].quantile(0.25)  
Q3 = df["Sales"].quantile(0.75)  
IQR = Q3 - Q1  
lower_limit = Q1 - 1.5 * IQR  
upper_limit = Q3 + 1.5 * IQR  
sales_outliers = df[  
    (df["Sales"] < lower_limit) |  
    (df["Sales"] > upper_limit)  
]  
print("\nSALES OUTLIER ANALYSIS")  
print("Q1:", Q1)  
print("Q3:", Q3)  
print("IQR:", IQR)  
print("Lower Limit:", lower_limit)  
print("Upper Limit:", upper_limit)  
print("\nSales Outliers:")  
print(  
    sales_outliers[  
        [  
            "Month",  
            "Sales"  
        ]  
    ]  
)  
# =====  
# COMMAND 25: Count Sales Outliers  
# =====  
print(  
    "\nNumber of Sales Outliers:",  
    len(sales_outliers)  
)  
# =====  
# COMMAND 26: Find Highest and Lowest Sales Month  
# =====  
highest_sales = df.loc[  
    df["Sales"].idxmax()  
]  
lowest_sales = df.loc[  
    df["Sales"].idxmin()  
]  
  
print("\nHIGHEST SALES MONTH")  
print(highest_sales[  
    [  
        "Month",  
        "Sales"  
    ]  
)
```



```

print("\nLOWEST SALES MONTH")
print(lowest_sales[
    [
        "Month",
        "Sales"
    ]
])
# =====
# COMMAND 27: Find Highest Advertising Month
# =====
highest_advertising = df.loc[
    df["Advertising_Expenditure"].idxmax()
]
print("\nHIGHEST ADVERTISING EXPENDITURE")
print(highest_advertising[
    [
        "Month",
        "Advertising_Expenditure"
    ]
])
# =====
# COMMAND 28: Find Highest Customer Visit Month
# =====
highest_visits = df.loc[
    df["Customer_Visits"].idxmax()
]
print("\nHIGHEST CUSTOMER VISITS")
print(highest_visits[
    [
        "Month",
        "Customer_Visits"
    ]
])
# =====
# COMMAND 29: Find Highest Product Returns Month
# =====
highest_returns = df.loc[
    df["Product_Returns"].idxmax()
]
print("\nHIGHEST PRODUCT RETURNS")
print(highest_returns[
    [
        "Month",
        "Product_Returns"
    ]
])
# =====
# COMMAND 30: Interactive Monthly Sales Visualization
# =====
fig = px.line(

```

```

    df,
    x="Month",
    y="Sales",
    markers=True,
    title="Interactive Monthly Sales Trend"
)
fig.show()
# =====
# COMMAND 31: Interactive Sales vs Advertising
# =====
fig = px.scatter(
    df,
    x="Advertising_Expenditure",
    y="Sales",
    size="Customer_Visits",
    color="Product_Returns",
    hover_data=["Month"],
    title="Interactive Business Relationship Analysis"
)
fig.show()
# =====
# COMMAND 32: Interactive Time-Based Business Analysis
# =====
fig = px.line(
    df,
    x="Month",
    y=[
        "Sales",
        "Advertising_Expenditure"
    ],
    markers=True,
    title="Interactive Sales and Advertising Trends"
)
fig.show()
# =====
# COMMAND 33: Final Findings
# =====
print("\n=====")
print("FINAL ANALYSIS SUMMARY")
print("=====")
print(
    "\n1. The dataset contains",
    len(df),
    "monthly observations."
)
print(
    "\n2. Average monthly sales:",
    round(df["Sales"].mean(), 2)
)
print(

```

```

        "\n3. Average advertising expenditure:",
        round(df["Advertising_Expenditure"].mean(), 2)
    )
    print(
        "\n4. Average customer visits:",
        round(df["Customer_Visits"].mean(), 2)
    )
    print(
        "\n5. Average product returns:",
        round(df["Product_Returns"].mean(), 2)
    )
    print(
        "\n6. Number of sales outliers:",
        len(sales_outliers)
    )
    print(
        "\n7. Correlation between Sales and Advertising:",
        round(
            correlation.loc[
                "Sales",
                "Advertising_Expenditure"
            ],
            2
        )
    )
    print(
        "\n8. Correlation between Sales and Customer Visits:",
        round(
            correlation.loc[
                "Sales",
                "Customer_Visits"
            ],
            2
        )
    )
    print(
        "\n9. Correlation between Sales and Product Returns:",
        round(
            correlation.loc[
                "Sales",
                "Product_Returns"
            ],
            2
        )
    )
    print("\n=====")
    print("ANALYSIS COMPLETED")
    print("=====")
    Output:

```

	Month	Sales	Advertising_Expenditure	Customer_Visits	Product_Returns
0	2023-01-01	86007.90	18548.49	4199	125
1	2023-02-01	93303.96	16628.03	4056	103
2	2023-03-01	108435.61	20350.81	4244	147
3	2023-04-01	120777.68	21193.02	4594	160
4	2023-05-01	99401.17	16488.14	4501	124

	Month	Sales	Advertising_Expenditure	Customer_Visits	Product_Returns
31	2025-08-01	201365.26	32768.45	6339	289
32	2025-09-01	190773.90	33077.96	6642	246
33	2025-10-01	193083.84	33035.21	6507	232
34	2025-11-01	200602.26	36108.76	6474	227
35	2025-12-01	217051.40	37532.15	6605	276

DATASET SHAPE
(36, 5)

COLUMN NAMES
['Month', 'Sales', 'Advertising_Expenditure', 'Customer_Visits', 'Product_Returns']

DATASET INFORMATION

```
<class 'pandas.DataFrame'>  
RangeIndex: 36 entries, 0 to 35  
Data columns (total 5 columns):  
#   Column                                Non-Null Count  Dtype  
---  ---                                -
```

#	Column	Non-Null Count	Dtype
0	Month	36 non-null	datetime64[us]
1	Sales	36 non-null	float64
2	Advertising_Expenditure	36 non-null	float64
3	Customer_Visits	36 non-null	int64

3	Customer_Visits	36 non-null	int64
4	Product_Returns	36 non-null	int64

dtypes: datetime64[us](1), float64(2), int64(2)
memory usage: 1.5 KB

MISSING VALUES

MISSING VALUES	
Month	0
Sales	0
Advertising_Expenditure	0
Customer_Visits	0
Product_Returns	0

DUPLICATE RECORDS	
Number of duplicates: 0	

STATISTICAL SUMMARY					
	Month	Sales	Advertising_Expenditure	Customer_Visits	Product_Returns
count	36	36.000000	36.000000	36.000000	36.000000
mean	2024-06-16 02:40:00	156565.983611	27272.937222	5468.111111	203.083333
min	2023-01-01 00:00:00	86007.900000	16488.140000	4056.000000	103.000000
25%	2023-09-23 12:00:00	119669.657500	21784.847500	4828.500000	155.250000

50%	2024-06-16 00:00:00	156283.480000	26969.040000	5403.000000	197.500000
75%	2025-03-08 18:00:00	191351.385000	32304.080000	6182.500000	230.500000
max	2025-12-01 00:00:00	220565.780000	45240.830000	6642.000000	445.000000
std	NaN	39528.512728	6512.709417	770.316513	65.012691

MONTH COLUMN CONVERTED TO DATETIME

- 0 2023-01-01
- 1 2023-02-01
- 2 2023-03-01
- 3 2023-04-01
- 4 2023-05-01

Name: Month, dtype: datetime64[us]

BUSINESS STATISTICS
Total Sales: 5636375.41
Average Sales: 156565.98
Total Advertising Expenditure: 981825.74

Average Customer Visits: 5468.11
Total Product Returns: 7311

CORRELATION MATRIX					
	Sales	Advertising_Expenditure	Customer_Visits	Product_Returns	
Sales	1.000000	0.846517	0.944210	0.871855	
Advertising_Expenditure	0.846517	1.000000	0.872774	0.711472	
Customer_Visits	0.944210	0.872774	1.000000	0.804680	
Product_Returns	0.871855	0.711472	0.804680	1.000000	

SALES OUTLIER ANALYSIS
Q1: 119669.6575
Q3: 191351.385
IQR: 71681.72750000001
Lower Limit: 12147.066249999989
Upper Limit: 298873.97625

Sales Outliers:
Empty DataFrame
Columns: [Month, Sales]
Index: []

Number of Sales Outliers: 0

HIGHEST SALES MONTH
Month 2025-06-01 00:00:00
Sales 220565.78
Name: 29, dtype: object

LOWEST SALES MONTH
Month 2023-01-01 00:00:00
Sales 86007.9
Name: 0, dtype: object

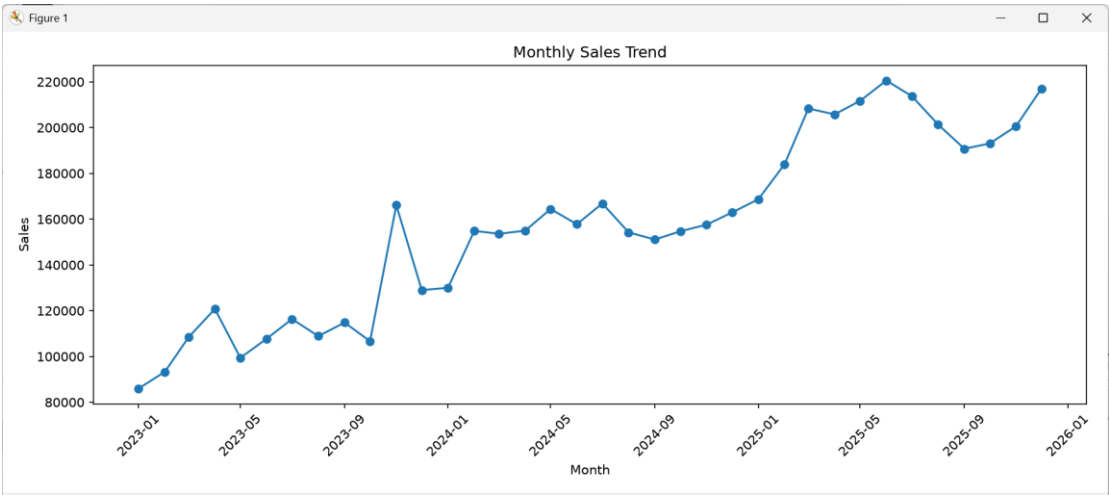
HIGHEST ADVERTISING EXPENDITURE
Month 2024-11-01 00:00:00
Advertising_Expenditure 45240.83
Name: 22, dtype: object

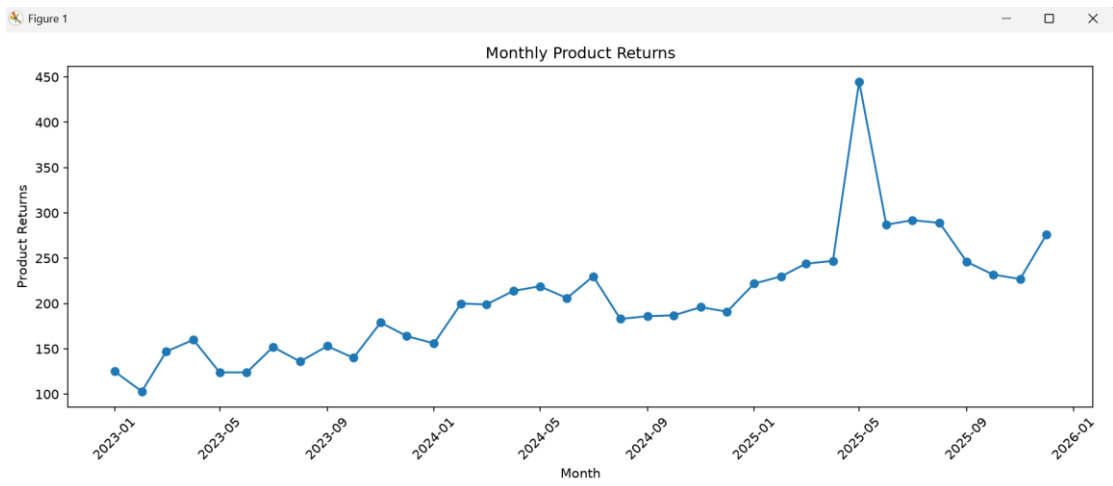
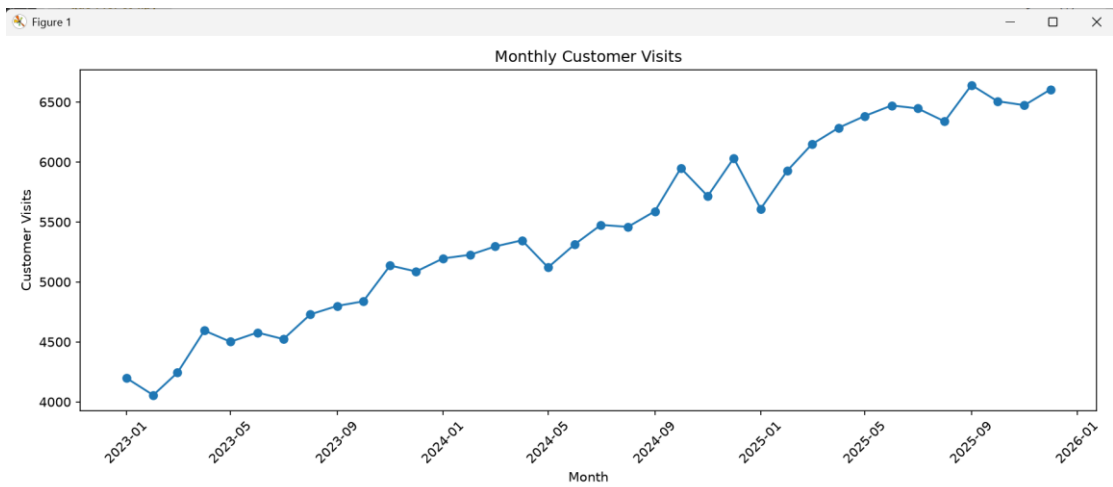
HIGHEST CUSTOMER VISITS	
Month	2025-09-01 00:00:00
Customer_Visits	6642
Name: 32, dtype: object	
HIGHEST PRODUCT RETURNS	
Month	2025-05-01 00:00:00
Product_Returns	445
Name: 28, dtype: object	

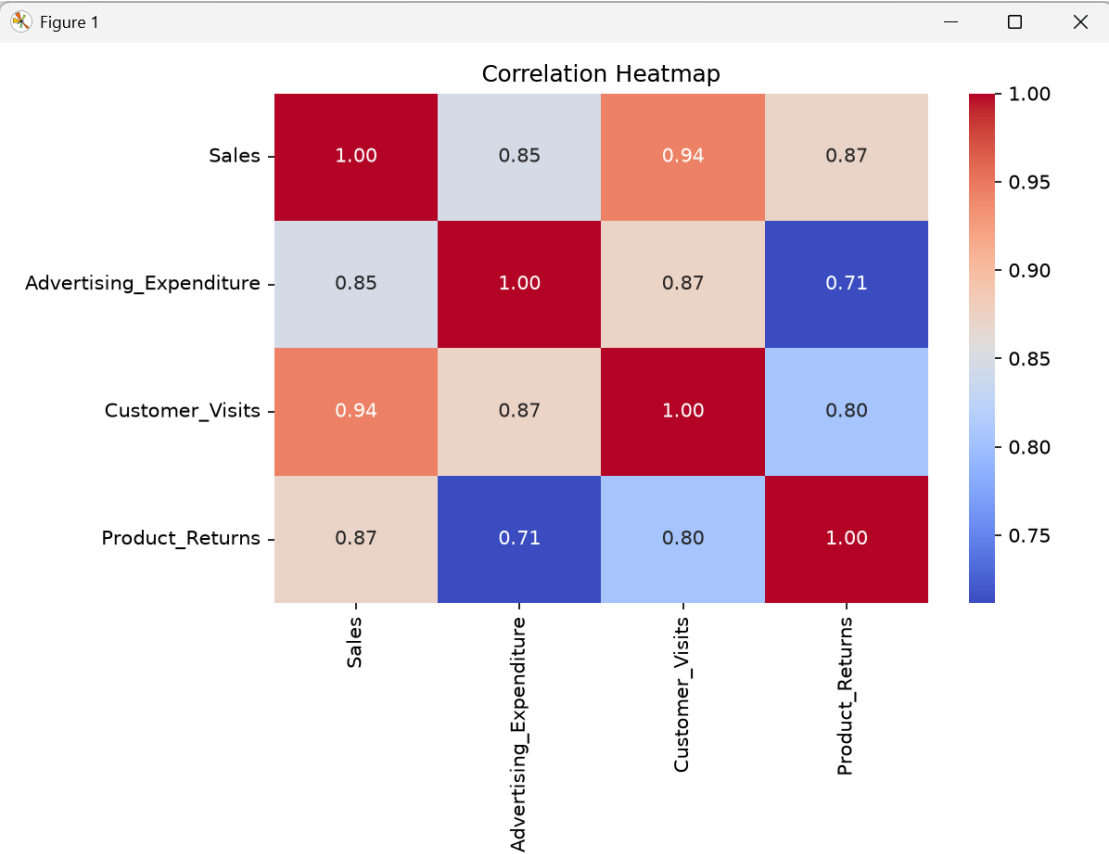
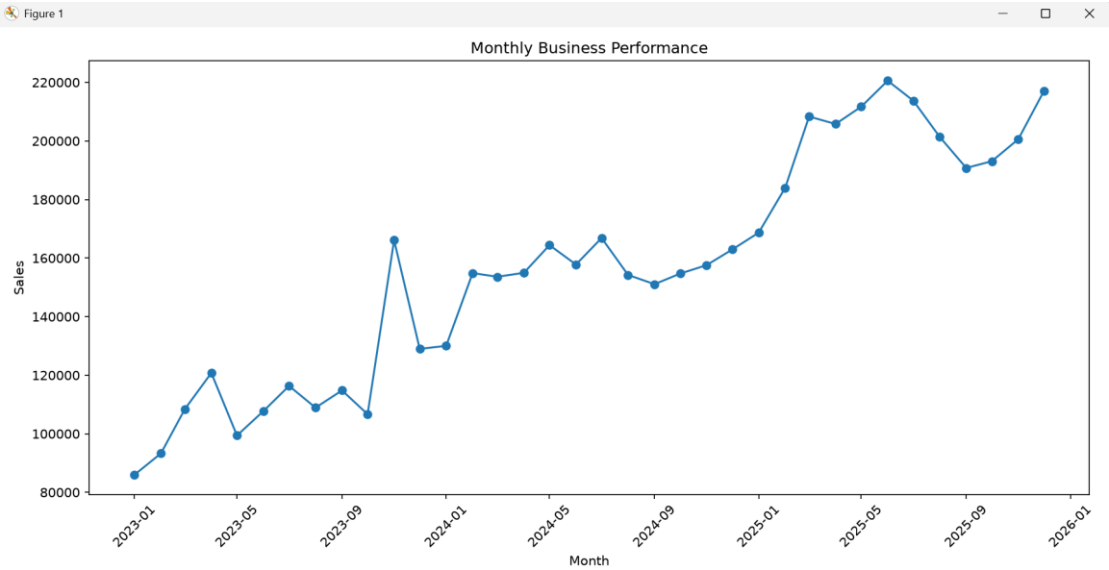
FINAL ANALYSIS SUMMARY

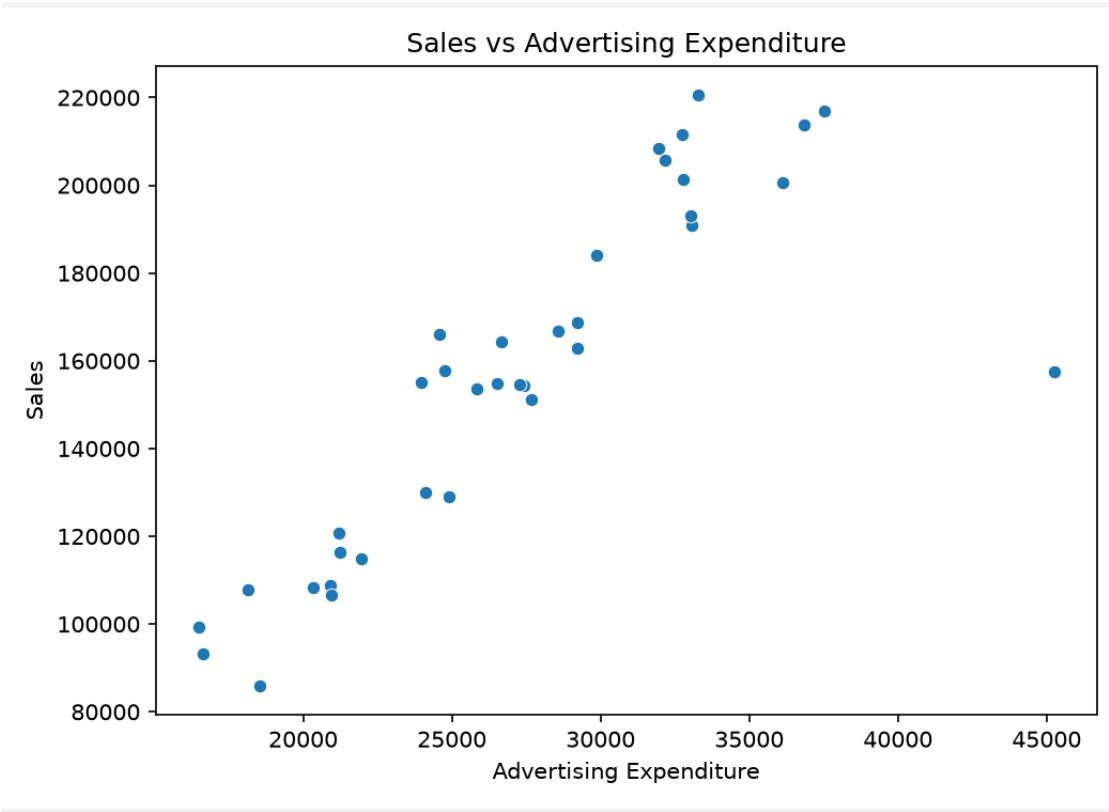
=====

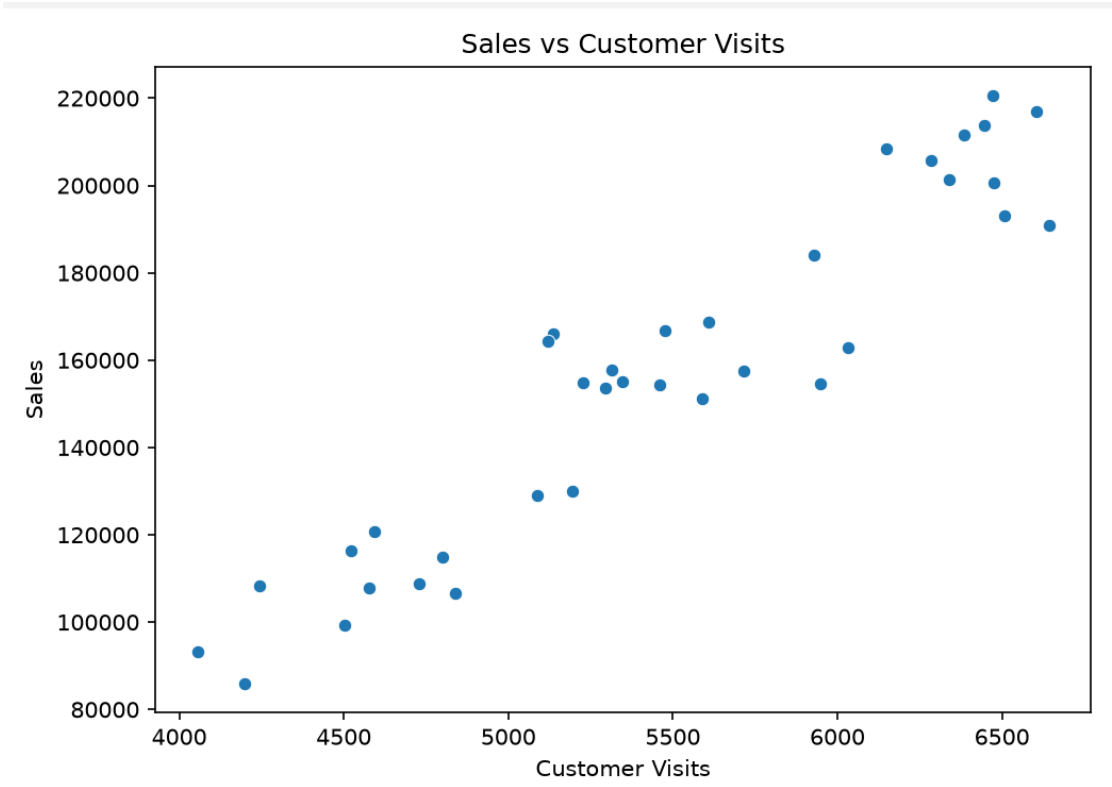
1. The dataset contains 36 monthly observations.
2. Average monthly sales: 156565.98
3. Average advertising expenditure: 27272.94
4. Average customer visits: 5468.11
5. Average product returns: 203.08
6. Number of sales outliers: 0
7. Correlation between Sales and Advertising: 0.85
8. Correlation between Sales and Customer Visits: 0.94
9. Correlation between Sales and Product Returns: 0.87

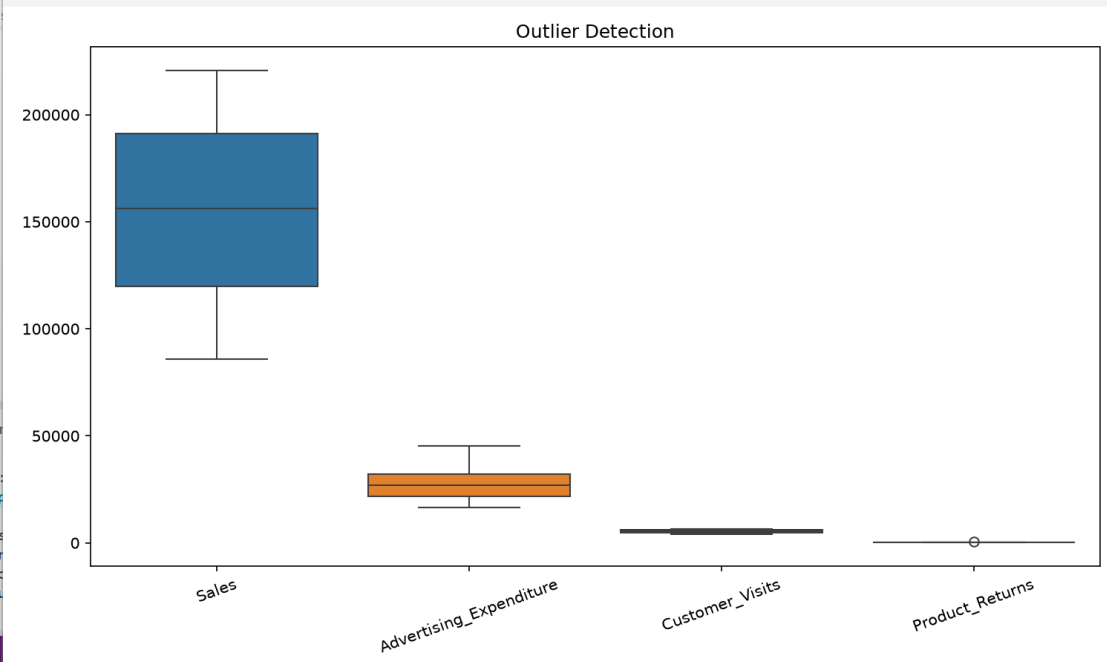




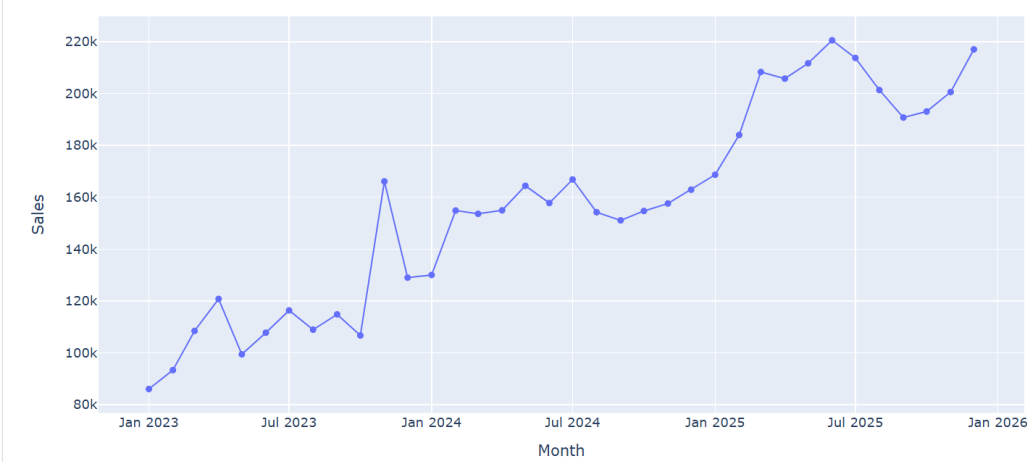




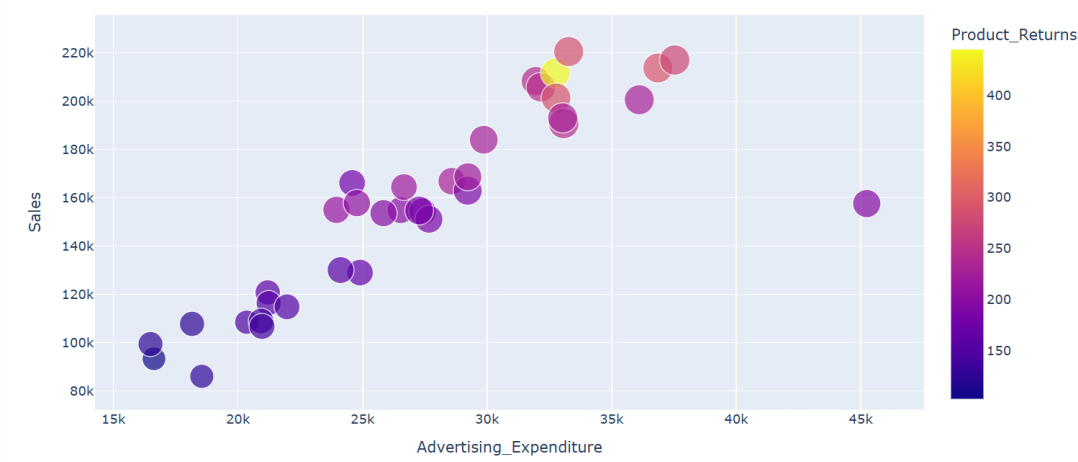




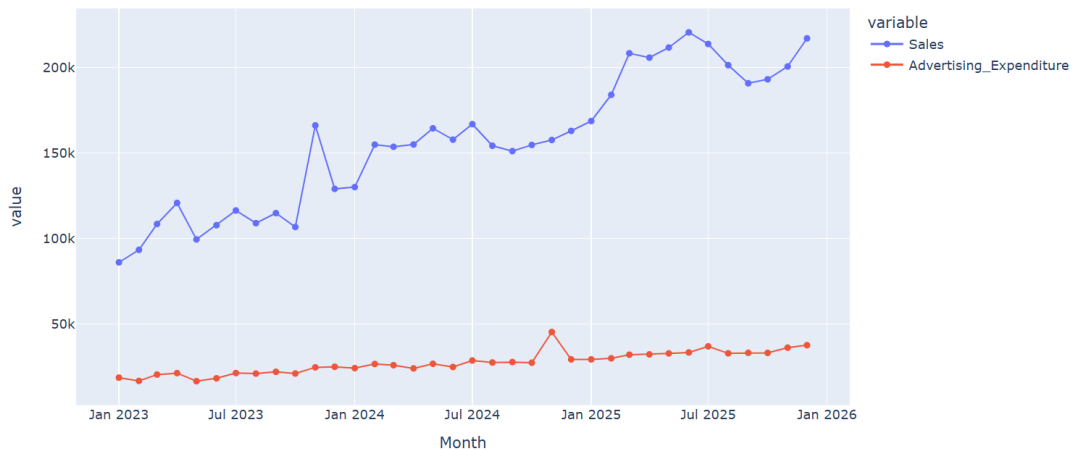
Interactive Monthly Sales Trend



Interactive Business Relationship Analysis



Interactive Sales and Advertising Trends



2. Multi-Dataset Business Pattern Analysis

A company provides separate datasets for **customers, products, sales, and regions**. Import and join the datasets, explore table functions, identify correlations among business variables, detect outliers, create meaningful groups, and focus on high-performing and low-performing segments. Present the discovered patterns using charts, interactive dashboards, and an open-source publishing platform.

Dataset	Purpose	Key Column
Customers	Customer information	Customer_ID
Products	Product information	Product_ID
Sales	Transaction information	Customer_ID,Product_ID,Region_ID
Regions	Regional information	Region_ID

```
# COMMAND 1: Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully")
# COMMAND 2: Import Four Datasets from Excel
file_path = "Q5_Multi_Dataset_Business_Pattern_Analysis.xlsx"
customers = pd.read_excel(
    file_path,
    sheet_name="Customers"
)
products = pd.read_excel(
    file_path,
    sheet_name="Products"
)
sales = pd.read_excel(
```

```

    file_path,
    sheet_name="Sales"
)
regions = pd.read_excel(
    file_path,
    sheet_name="Regions"
)
print("All four datasets imported successfully")
# COMMAND 3: Display First 5 Records
print("\n===== CUSTOMERS =====")
print(customers.head())
print("\n===== PRODUCTS =====")
print(products.head())
print("\n===== SALES =====")
print(sales.head())
print("\n===== REGIONS =====")
print(regions.head())
# COMMAND 4: Display Dataset Shapes
print("\n===== DATASET SHAPES =====")
print("Customers:", customers.shape)
print("Products :", products.shape)
print("Sales   :", sales.shape)
print("Regions  :", regions.shape)
#          COMMAND          5:          Display          Column          Names
=====
print("\n===== COLUMN NAMES =====")
print("Customer Columns:")
print(customers.columns.tolist())
print("\nProduct Columns:")
print(products.columns.tolist())
print("\nSales Columns:")
print(sales.columns.tolist())
print("\nRegion Columns:")
print(regions.columns.tolist())
# COMMAND 6: Dataset Information
print("\n===== CUSTOMERS INFORMATION =====")
customers.info()
print("\n===== PRODUCTS INFORMATION =====")
products.info()
print("\n===== SALES INFORMATION =====")
sales.info()
print("\n===== REGIONS INFORMATION =====")
regions.info()
=====
# COMMAND 7: Check Missing Values
print("\n===== MISSING VALUES =====")
print("\nCustomers:")
print(customers.isnull().sum())
print("\nProducts:")
print(products.isnull().sum())

```

```

print("\nSales:")
print(sales.isnull().sum())

print("\nRegions:")
print(regions.isnull().sum())
# COMMAND 8: Check Duplicate Records
print("\n===== DUPLICATE RECORDS =====")
print("Customer duplicates:", customers.duplicated().sum())
print("Product duplicates :", products.duplicated().sum())
print("Sales duplicates  :", sales.duplicated().sum())
print("Region duplicates :", regions.duplicated().sum())
# COMMAND 9: Statistical Summary
print("\n===== CUSTOMER STATISTICS =====")
print(customers.describe())
print("\n===== PRODUCT STATISTICS =====")
print(products.describe())
print("\n===== SALES STATISTICS =====")
print(sales.describe())
# COMMAND 10: Convert Date Column
sales["Date"] = pd.to_datetime(sales["Date"])
print("\nDate column converted to datetime")
print(sales["Date"].head())
# COMMAND 11: JOIN SALES WITH CUSTOMERS
df = sales.merge(
    customers,
    on="Customer_ID",
    how="left"
)
print("\n===== AFTER CUSTOMER JOIN =====")
print(df.head())
print("Shape:", df.shape)
# COMMAND 12: JOIN WITH PRODUCTS
df = df.merge(
    products,
    on="Product_ID",
    how="left"
)
print("\n===== AFTER PRODUCT JOIN =====")
print(df.head())
print("Shape:", df.shape)
# COMMAND 13: JOIN WITH REGIONS
df = df.merge(
    regions,
    on="Region_ID",
    how="left"
)
print("\n===== AFTER REGION JOIN =====")
print(df.head())
print("Shape:", df.shape)

```

```

# COMMAND 14: Display Integrated Dataset
print("\n===== FINAL INTEGRATED DATASET =====")
print(df.head())
print("\nFinal Shape:")
print(df.shape)
print("\nFinal Columns:")
print(df.columns.tolist())
# COMMAND 15: Check Missing Values After Joining
print("\n===== MISSING VALUES AFTER JOIN =====")
print(df.isnull().sum())
# COMMAND 16: Total Business Sales
total_sales = df["Sales_Amount"].sum()
print("\n===== BUSINESS SALES =====")
print(
    "Total Sales:",
    round(total_sales, 2)
)
# COMMAND 17: Average Transaction Sales
average_sales = df["Sales_Amount"].mean()
print(
    "Average Transaction Sales:",
    round(average_sales, 2)
)
# COMMAND 18: Total Quantity Sold
total_quantity = df["Quantity"].sum()
print(
    "Total Quantity Sold:",
    total_quantity
)
# COMMAND 19: Sales by Region
region_sales = df.groupby(
    "Region_Name"
)["Sales_Amount"].sum().sort_values(
    ascending=False
)
print("\n===== SALES BY REGION =====")
print(region_sales)
# COMMAND 20: Visualize Sales by Region
plt.figure(figsize=(10, 5))
region_sales.plot(kind="bar")
plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# COMMAND 21: Sales by Product Category
category_sales = df.groupby(
    "Category"
)["Sales_Amount"].sum().sort_values(

```

```

    ascending=False
)
print("\n===== SALES BY CATEGORY =====")
print(category_sales)
# COMMAND 22: Visualize Category Sales
plt.figure(figsize=(10, 5))
category_sales.plot(kind="bar")
plt.title("Sales by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# COMMAND 23: Top 10 Products
product_sales = df.groupby(
    "Product_Name"
)["Sales_Amount"].sum().sort_values(
    ascending=False
)
print("\n===== TOP 10 PRODUCTS =====")
print(product_sales.head(10))
# COMMAND 24: Visualize Top 10 Products
plt.figure(figsize=(10, 5))
product_sales.head(10).plot(kind="bar")
plt.title("Top 10 Products by Sales")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# COMMAND 25: Top 10 Customers
customer_sales = df.groupby(
    "Customer_Name"
)["Sales_Amount"].sum().sort_values(
    ascending=False
)
print("\n===== TOP 10 CUSTOMERS =====")
print(customer_sales.head(10))
# COMMAND 26: Visualize Top 10 Customers
plt.figure(figsize=(10, 5))
customer_sales.head(10).plot(kind="bar")
plt.title("Top 10 Customers by Sales")
plt.xlabel("Customer")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# COMMAND 27: Sales by Customer Category
customer_category_sales = df.groupby(
    "Customer_Category"

```



```

)["Sales_Amount"].sum().sort_values(
    ascending=False
)
print("\n===== SALES BY CUSTOMER CATEGORY =====")
print(customer_category_sales)
# COMMAND 28: Visualize Customer Category Sales
plt.figure(figsize=(8, 5))
customer_category_sales.plot(kind="bar")
plt.title("Sales by Customer Category")
plt.xlabel("Customer Category")
plt.ylabel("Sales")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
# COMMAND 29: Correlation Analysis
business_columns = [
    "Quantity",
    "Sales_Amount",
    "Discount",
    "Unit_Price",
    "Cost"
]
correlation = df[business_columns].corr()
print("\n===== CORRELATION MATRIX =====")
print(correlation)
# COMMAND 30: Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Business Variable Correlation")
plt.tight_layout()
plt.show()
# COMMAND 31: Quantity vs Sales
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x="Quantity",
    y="Sales_Amount"
)
plt.title("Quantity vs Sales Amount")
plt.xlabel("Quantity")
plt.ylabel("Sales Amount")
plt.tight_layout()
plt.show()
=====
# COMMAND 32: Discount vs Sales

```

```

# =====
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x="Discount",
    y="Sales_Amount"
)
plt.title("Discount vs Sales Amount")
plt.xlabel("Discount (%)")
plt.ylabel("Sales Amount")
plt.tight_layout()
plt.show()
=====

# COMMAND 33: Unit Price vs Sales
# =====
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x="Unit_Price",
    y="Sales_Amount"
)
plt.title("Unit Price vs Sales Amount")
plt.xlabel("Unit Price")
plt.ylabel("Sales Amount")
plt.tight_layout()
plt.show()
=====

# COMMAND 34: Outlier Detection Using Box Plot
=====

plt.figure(figsize=(8, 5))

sns.boxplot(
    y=df["Sales_Amount"]
)
plt.title("Sales Amount Outlier Detection")
plt.ylabel("Sales Amount")
plt.tight_layout()
plt.show()
=====

# COMMAND 35: IQR Outlier Detection
=====

Q1 = df["Sales_Amount"].quantile(0.25)
Q3 = df["Sales_Amount"].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
outliers = df[
    (df["Sales_Amount"] < lower_limit) |
    (df["Sales_Amount"] > upper_limit)
]

```

```

print("\n===== OUTLIER ANALYSIS =====")
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
print("\nNumber of Outliers:")
print(len(outliers))

=====
# COMMAND 36: Display Outlier Transactions
=====
print("\n===== OUTLIER TRANSACTIONS =====")
print(
    outliers[
        [
            "Sale_ID",
            "Customer_Name",
            "Product_Name",
            "Region_Name",
            "Sales_Amount"
        ]
    ]
)

=====
# COMMAND 37: Create High and Low Performance Groups
=====
median_sales = df["Sales_Amount"].median()
df["Performance"] = np.where(
    df["Sales_Amount"] >= median_sales,
    "High Performing",
    "Low Performing"
)
print("\n===== PERFORMANCE GROUP =====")
print(
    df[
        [
            "Sale_ID",
            "Sales_Amount",
            "Performance"
        ]
    ].head(10)
)

=====
# COMMAND 38: Count High and Low Performing Transactions
=====
performance_count = df[
    "Performance"
].value_counts()
print("\n===== PERFORMANCE COUNTS =====")
print(performance_count)

```

```

=====
# COMMAND 39: Visualize High vs Low Performance
# =====
plt.figure(figsize=(8, 5))
performance_count.plot(kind="bar")
plt.title("High vs Low Performing Transactions")
plt.xlabel("Performance")
plt.ylabel("Number of Transactions")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
=====

# COMMAND 40: High-Performing Regions
=====
high_region = df[
    df["Performance"] == "High Performing"
].groupby(
    "Region_Name"
)["Sales_Amount"].sum().sort_values(
    ascending=False
)
print("\n===== HIGH-PERFORMING REGIONS =====")
print(high_region)
=====

# COMMAND 41: Low-Performing Regions
=====
low_region = df[
    df["Performance"] == "Low Performing"
].groupby(
    "Region_Name"
)["Sales_Amount"].sum().sort_values()
print("\n===== LOW-PERFORMING REGIONS =====")
print(low_region)
=====

# COMMAND 42: High-Performing Products
=====
high_products = df[
    df["Performance"] == "High Performing"
].groupby(
    "Product_Name"
)["Sales_Amount"].sum().sort_values(
    ascending=False
)
print("\n===== HIGH-PERFORMING PRODUCTS =====")
print(high_products.head(10))
=====

# COMMAND 43: Low-Performing Products
=====
low_products = df[
    df["Performance"] == "Low Performing"

```

```

].groupby(
    "Product_Name"
)["Sales_Amount"].sum().sort_values()
print("\n===== LOW-PERFORMING PRODUCTS =====")
print(low_products.head(10))
=====

# COMMAND 44: Monthly Sales Analysis
=====

monthly_sales = df.groupby(
    df["Date"].dt.to_period("M")
)["Sales_Amount"].sum()
monthly_sales.index = monthly_sales.index.astype(str)
print("\n===== MONTHLY SALES =====")
print(monthly_sales)
=====

# COMMAND 45: Monthly Sales Visualization
=====

plt.figure(figsize=(12, 5))
plt.plot(
    monthly_sales.index,
    monthly_sales.values,
    marker="o"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
=====

# COMMAND 46: Region vs Category Analysis
=====

region_category = pd.pivot_table(
    df,
    values="Sales_Amount",
    index="Region_Name",
    columns="Category",
    aggfunc="sum"
)
print("\n===== REGION VS CATEGORY =====")
print(region_category)
=====

# COMMAND 47: Region-Category Heatmap
=====

plt.figure(figsize=(12, 6))
sns.heatmap(
    region_category,
    annot=True,
    fmt=".0f",
    cmap="YlGnBu"
)

```

```
)
plt.title("Sales by Region and Product Category")
plt.tight_layout()
plt.show()
```

```
=====
# COMMAND 48: Interactive Visualization
=====
```

```
import plotly.express as px
fig = px.scatter(
    df,
    x="Quantity",
    y="Sales_Amount",
    color="Region_Name",
    size="Discount",
    hover_data=[
        "Customer_Name",
        "Product_Name",
        "Category",
        "Customer_Category"
    ],
    title="Interactive Business Pattern Analysis"
)
fig.show()
```

```
=====
# COMMAND 49: Interactive Regional Sales
=====
```

```
region_chart = df.groupby(
    "Region_Name",
    as_index=False
)["Sales_Amount"].sum()
fig = px.bar(
    region_chart,
    x="Region_Name",
    y="Sales_Amount",
    color="Region_Name",
    title="Interactive Regional Sales"
)
fig.show()
```

```
=====
# COMMAND 50: Interactive Product Category Sales
=====
```

```
category_chart = df.groupby(
    "Category",
    as_index=False
)["Sales_Amount"].sum()
fig = px.bar(
    category_chart,
    x="Category",
    y="Sales_Amount",
    color="Category",
```

```

        title="Interactive Product Category Sales"
    )
    fig.show()
=====
# COMMAND 51: Interactive Monthly Sales
=====
monthly_interactive = df.groupby(
    df["Date"].dt.to_period("M"),
    as_index=False
)["Sales_Amount"].sum()
monthly_interactive["Date"] = (
    monthly_interactive["Date"].astype(str)
)
fig = px.line(
    monthly_interactive,
    x="Date",
    y="Sales_Amount",
    markers=True,
    title="Interactive Monthly Sales Trend"
)
fig.show()
=====
# COMMAND 52: Final Business Summary
# =====
print("\n")
print("=====")
print("      FINAL BUSINESS ANALYSIS")
print("=====")
print(
    "\nTotal Sales:",
    round(df["Sales_Amount"].sum(), 2)
)
print(
    "Average Transaction Sales:",
    round(df["Sales_Amount"].mean(), 2)
)
print(
    "Total Quantity Sold:",
    int(df["Quantity"].sum())
)
print(
    "Number of Transactions:",
    len(df)
)
print(
    "Number of Customers:",
    df["Customer_ID"].nunique()
)
print(
    "Number of Products:",

```

```

    df["Product_ID"].nunique()
)
print(
    "Number of Regions:",
    df["Region_ID"].nunique()
)
print(
    "Number of Outliers:",
    len(outliers)
)
=====
# COMMAND 53: Best Performing Region
=====
best_region = region_sales.idxmax()
best_region_sales = region_sales.max()
print(
    "\nBest Performing Region:",
    best_region
)
print(
    "Sales:",
    round(best_region_sales, 2)
)
=====
# COMMAND 54: Lowest Performing Region
=====
lowest_region = region_sales.idxmin()
lowest_region_sales = region_sales.min()
print(
    "\nLowest Performing Region:",
    lowest_region
)
print(
    "Sales:",
    round(lowest_region_sales, 2)
)
=====
# COMMAND 55: Best Product Category
=====
best_category = category_sales.idxmax()
best_category_sales = category_sales.max()
print(
    "\nBest Product Category:",
    best_category
)
print(
    "Sales:",
    round(best_category_sales, 2)
)
=====

```



```

# COMMAND 56: Best Product
=====
best_product = product_sales.idxmax()
best_product_sales = product_sales.max()
print(
    "\nBest Product:",
    best_product
)
print(
    "Sales:",
    round(best_product_sales, 2)
)
=====
# COMMAND 57: Best Customer
=====
best_customer = customer_sales.idxmax()
best_customer_sales = customer_sales.max(
print(
    "\nBest Customer:",
    best_customer
)
print(
    "Sales:",
    round(best_customer_sales, 2)
)
=====
# COMMAND 58: Strongest Correlations with Sales
=====
sales_correlation = correlation[
    "Sales_Amount"
].drop("Sales_Amount").sort_values(
    ascending=False
print("\n===== CORRELATION WITH SALES =====")
print(sales_correlation)
=====
# COMMAND 59: Save Integrated Dataset
# =====
df.to_excel(
    "Q5_Integrated_Business_Dataset.xlsx",
    index=False
)
print(
    "\nIntegrated dataset saved as:",
    "Q5_Integrated_Business_Dataset.xlsx"
)
=====
# COMMAND 60: FINAL MESSAGE
# =====
print("\n")
print("=====")

```

```
print("Q5 ANALYSIS COMPLETED SUCCESSFULLY")
print("=====")
```

Datasets imported successfully

Customers: (120, 5)

Products: (30, 5)

Sales: (600, 8)

Regions: (8, 4)

	Customer_ID	Customer_Name	Age	Gender	Customer_Category
0	C001	Customer 1	63	Male	Regular
1	C002	Customer 2	33	Male	Regular
2	C003	Customer 3	44	Male	Regular
3	C004	Customer 4	42	Female	Regular
4	C005	Customer 5	58	Female	Regular

	Product_ID	Product_Name	Category	Unit_Price	Cost
0	P001	Product 1	Electronics	68923.92	53873.18
1	P002	Product 2	Beauty	58706.41	44583.77
2	P003	Product 3	Sports	8738.84	7003.25
3	P004	Product 4	Sports	74779.08	42029.24
4	P005	Product 5	Fashion	66000.40	40295.53

	Sale_ID	Customer_ID	Product_ID	Region_ID	Date	Quantity	Sales_Amount	Discount
0	S0001	C065	P003	R06	2026-04-01	9	58221.04	21.27
1	S0002	C084	P001	R08	2025-03-09	5	282452.83	21.21
2	S0003	C031	P028	R05	2026-04-06	9	40484.15	5.69
3	S0004	C078	P006	R04	2025-08-13	2	44285.26	1.26
4	S0005	C054	P025	R04	2025-12-22	5	249686.33	21.69

Missing Values:

Sale_ID 0

Customer_ID 0

Product_ID 0

Region_ID 0

Date 0

	Sale_ID	Customer_ID	Product_ID	Region_ID	Date	Quantity	...	Category	Unit_Price	Cost	Region_Name	State	Region_Type
0	S0001	C065	P003	R06	2026-04-01	9	...	Sports	8738.84	7003.25	Northeast	Assam	Urban
1	S0002	C084	P001	R08	2025-03-09	5	...	Electronics	68923.92	53873.18	Southwest	Gujarat	Semi-Urban
2	S0003	C031	P028	R05	2026-04-06	9	...	Fashion	4763.78	3740.64	Central	Madhya Pradesh	Semi-Urban
3	S0004	C078	P006	R04	2025-08-13	2	...	Electronics	21651.18	12719.57	West	Maharashtra	Rural
4	S0005	C054	P025	R04	2025-12-22	5	...	Sports	60210.39	36430.90	West	Maharashtra	Rural

[5 rows x 19 columns]

Final Shape: (600, 19)

Sales by Region:

Region_Name

West 1.735316e+07

West	1.735316e+07
East	1.496589e+07
South	1.428848e+07
Southeast	1.375236e+07
North	1.248086e+07
Southwest	1.210707e+07

North	1.248086e+07
Southwest	1.210707e+07
Central	1.014700e+07
Northeast	8.961346e+06

Name: Sales_Amount, dtype: float64

Sales by Category:

Category

Fashion	2.715973e+07
Electronics	2.605861e+07
Home Appliances	2.125788e+07
Sports	1.931323e+07

Beauty	1.026673e+07
--------	--------------

Name: Sales_Amount, dtype: float64

Correlation Matrix:

	Quantity	Sales_Amount	Discount	Unit_Price	Cost
Quantity	1.000000	0.549700	0.030915	0.038752	0.045748
Sales_Amount	0.549700	1.000000	-0.064804	0.733129	0.720153
Discount	0.030915	-0.064804	1.000000	-0.039299	-0.033807
Unit_Price	0.038752	0.733129	-0.039299	1.000000	0.979356
Cost	0.045748	0.720153	-0.033807	0.979356	1.000000

Number of Outliers: 1

Performance Groups:

Performance

Low Performing	300
High Performing	300

Name: count, dtype: int64

High Performing Regions:

Region_Name

West	1.572456e+07
East	1.360232e+07
South	1.281994e+07
Southeast	1.194236e+07
North	1.068483e+07
Southwest	1.011894e+07
Central	8.798061e+06
Northeast	7.530311e+06

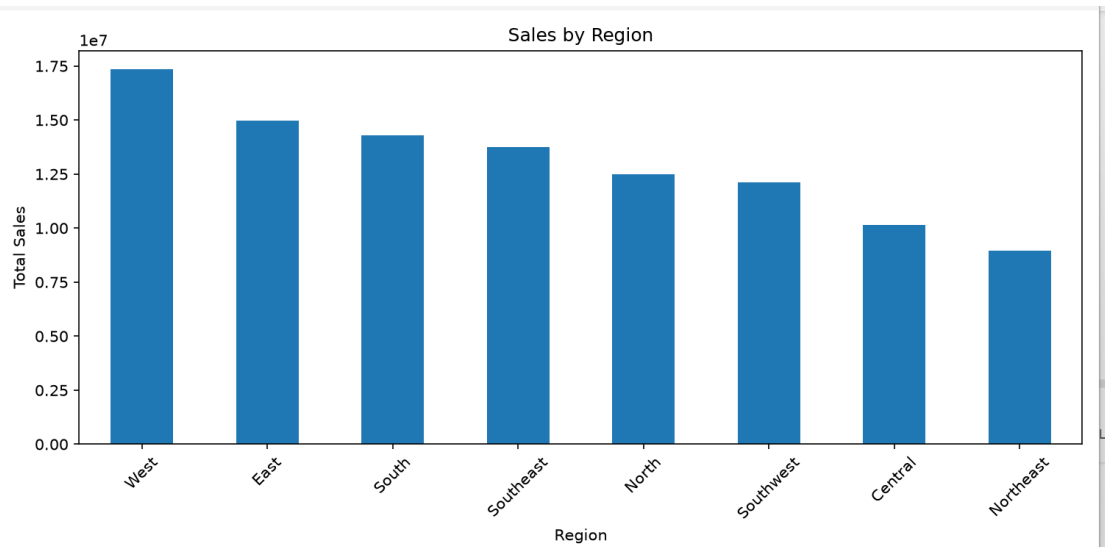
Name: Sales_Amount, dtype: float64

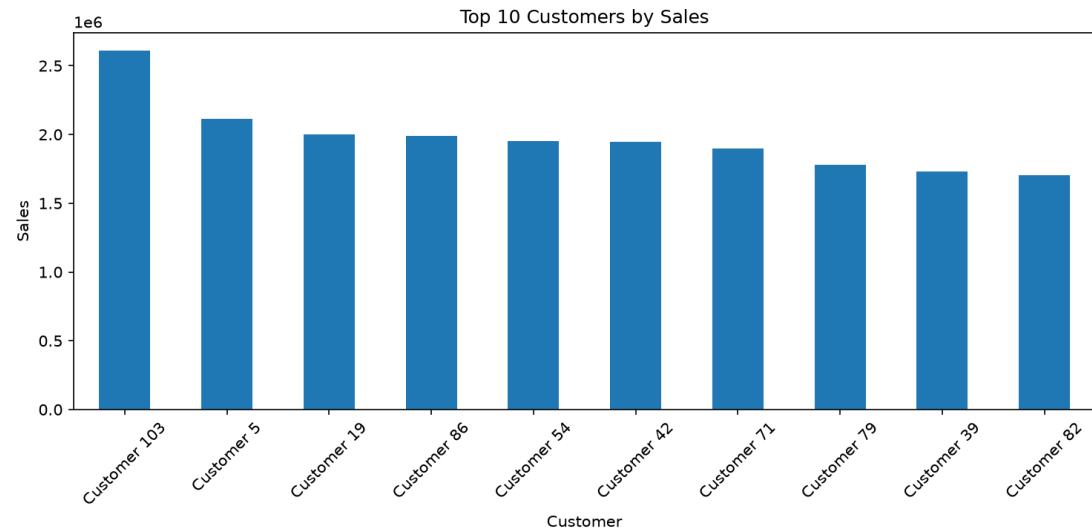
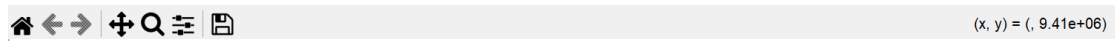
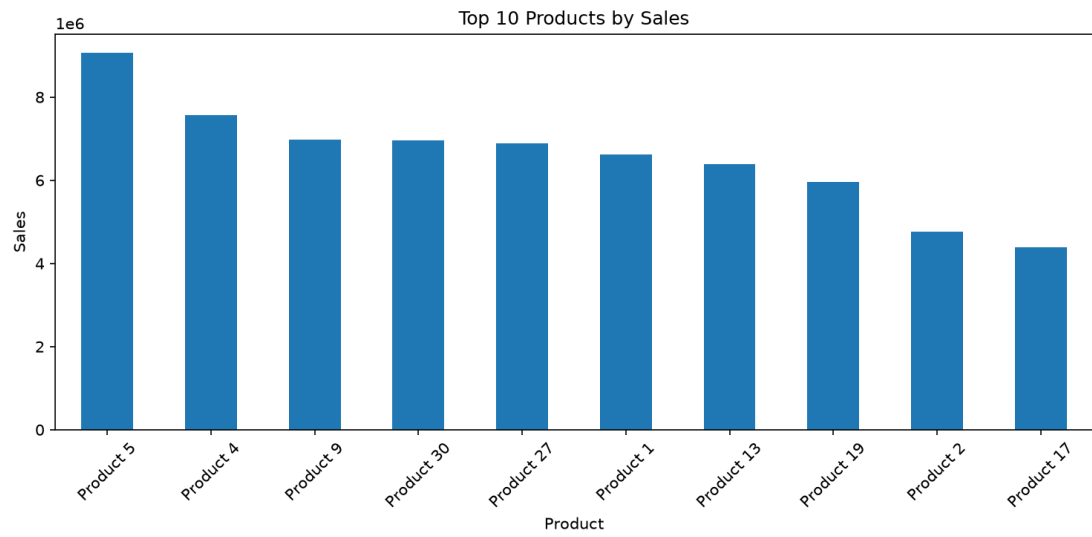
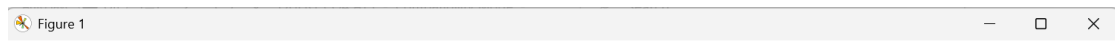
Low Performing Regions:

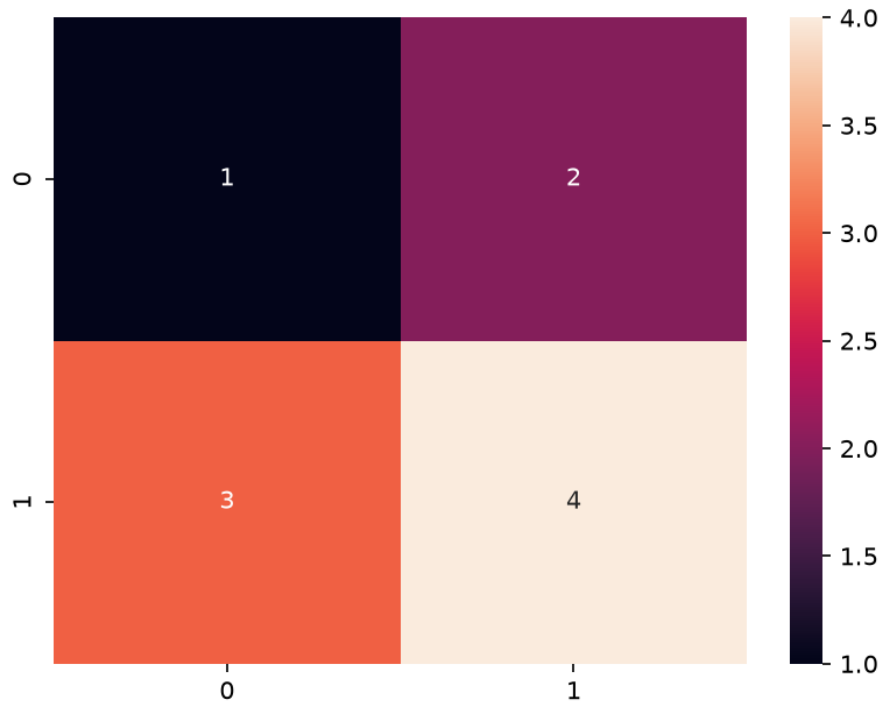
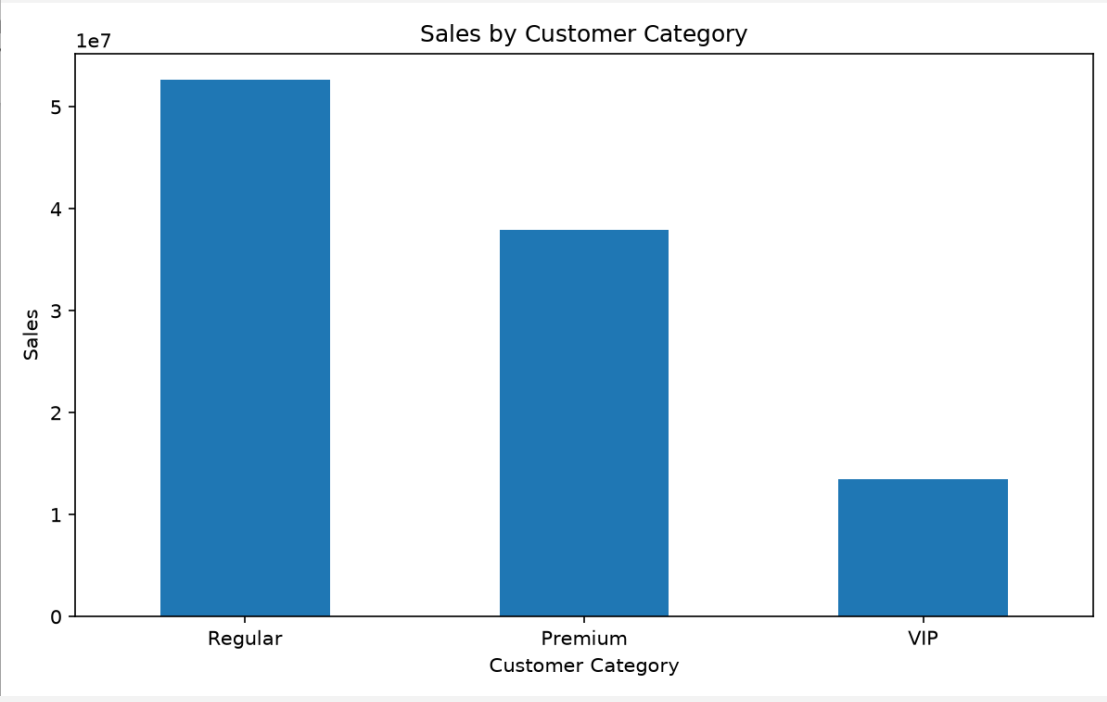
Region_Name	
Central	1348943.03
East	1363575.48
Northeast	1431034.94
South	1468548.09

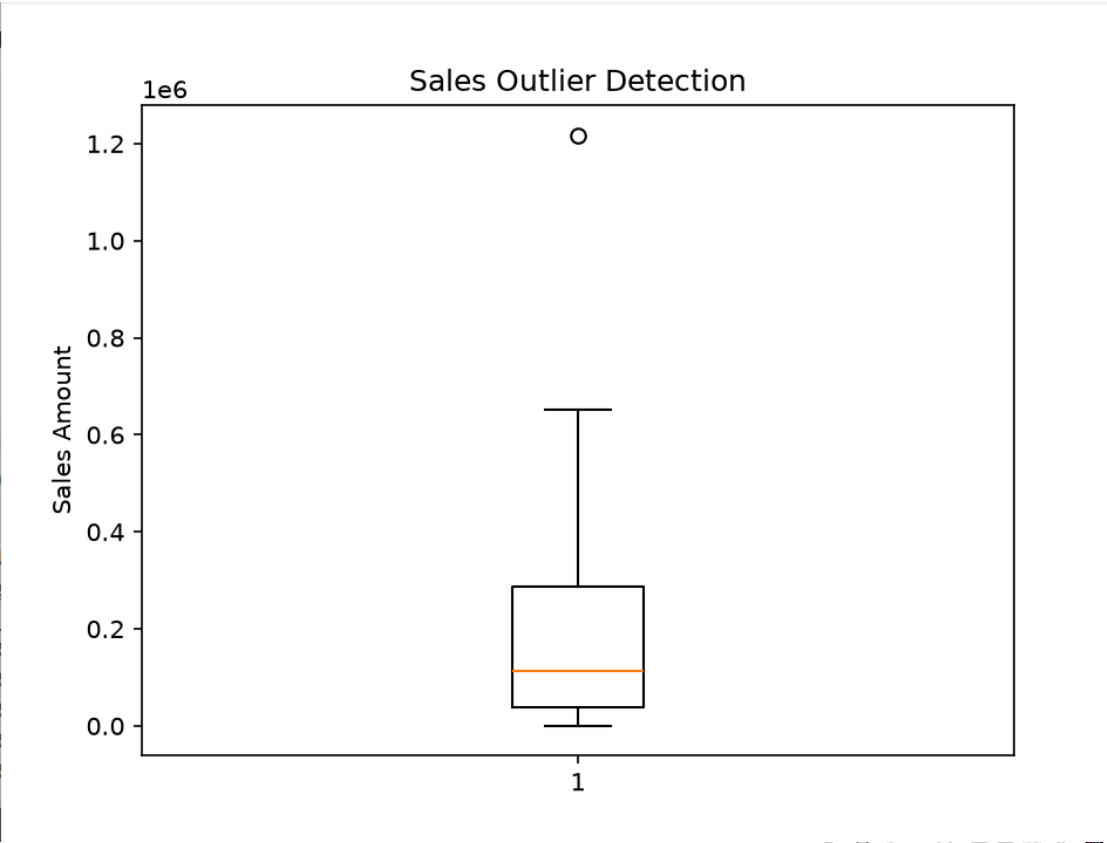
West	1628601.22
North	1796031.12
Southeast	1810002.40
Southwest	1988127.67

Name: Sales_Amount, dtype: float64









Interactive Business Pattern Analysis

